COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	September 30, 2026	**DATE SUBMITTED:**	September 30, 2026
NAME:	Silao, Johrel Louie T. Subalisid, Jappeth Royce Villaruel, Prince Benjamin	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

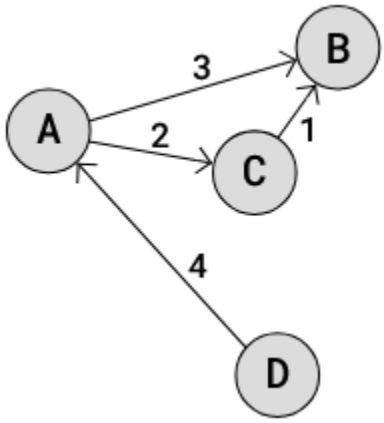
ACTIVITY NO. 10.1 Graph

1. PROCEDURE OUTPUT

1. Define the following:

1. Graph
   - A visual diagram that shows how different items or numbers relate to each other.
2. Vertex
   - It is a fundamental part of a graph that contains additional information which represents an entity.
3. Nodes
   - A basic building block that is used to construct node based data structures like linked lists, trees and graphs.
4. Weight
   - A variable name that is used to store a numerical value assigned to an edge in a weighted graph.
5. Path
   - A sequence of vertices that are connected by edges.
6. Directed graph
   - A type of graph where all edges in a graph are all one-way.
7. Connected graph
   - It is a type of graph where there is a path between every single pair of nodes.
8. Directed cyclic
   - A path that starts and ends at the same vertex.
9. Adjacency matrix
   - It is a two-dimensional matrix where each cell indicates that there is an edge from vertex to vertex.
10. Adjacency List
    - It is a collection of all the vertices in the graph object. Each vertex object in the graph keeps a list of the other vertices to which it is connected.

2. Identify the parts of the following graph.



A. **Nodes:** {A, B, C, D}
B. **Edge and Weight:**

Edge	Weight
$A \rightarrow B$	3
$A \rightarrow C$	2
$C \rightarrow B$	1
$D \rightarrow A$	4

C. **Type of Graph:** Directed Graph (Digraph)

3. Plot the adjacency matrix of the graph above. (10 pts)

A->B (weight 3)
A->C (weight 2)
C->B (weight 1)
D->A (weight 4)

From/to	A	B	C	D
A	0	3	2	0
B	0	0	0	0
C	0	1	0	0
D	4	0	0	0

4. Create a program in C++ program to create a graph that looks like in the above figure.

```
#include <iostream>

const int maxChildren = 10;

template <typename T>
class Graph {
    private:
        struct Edge {
            Graph<T>* target = nullptr;
            int weight = 0;
        };
        T data;
        Edge children[maxChildren];
    public:
        Graph(T newData);
        void connect(Graph<T>& receiverNode, int weight);
        void print();
};

template <typename T>
Graph<T>::Graph(T newData) {
    data = newData;
}

template <typename T>
void Graph<T>::connect(Graph<T>& receiverNode, int weight) {
    for(int i = 0; i < maxChildren; i++) {//loop thru array of children
        if(children[i].target == nullptr) {//insert at first nullpointer
            children[i].target = &receiverNode;
            children[i].weight = weight;
            return;
        }
    }
    std::cout << "\nThere's no more children slot\n"; //print if didn't
return
}
```

```cpp
template <typename T>
void Graph<T>::print() {
    std::cout << "Node " << data << " connects to: ";
    bool hasEdges = false;

    for (int i = 0; i < maxChildren; i++) {
        if (children[i].target != nullptr) {
            std::cout << std::endl << children[i].target->data
                      << " (Weight " << children[i].weight << ")";
            hasEdges = true;
        }
    }

    if (!hasEdges) {
        std::cout << "\nNothing";
    }
}


int main() {
    Graph<char> nodeA('A');
    Graph<char> nodeB('B');
    Graph<char> nodeC('C');
    Graph<char> nodeD('D');


    nodeD.connect(nodeA, 4);
    nodeA.connect(nodeB, 3);
    nodeA.connect(nodeC, 2);
    nodeC.connect(nodeB, 1);

    nodeA.print();
    std::cout << std::endl;
    nodeB.print();
    std::cout << std::endl;
    nodeC.print();
    std::cout << std::endl;
```

```
    nodeD.print();


}
```

```
Node A connects to:
B (Weight 3)
C (Weight 2)
Node B connects to:
Nothing
Node C connects to:
B (Weight 1)
Node D connects to:
A (Weight 4)
```

5. Differentiate BFS vs DFS types of search.

- Breadth-First Search (BFS) and Depth-First Search (DFS) are two ways of traversing a graph. BFS explores level by level, visiting all neighbors of a vertex before moving deeper, and uses a queue (FIFO). DFS follows one path as far as it can, then backtracks to try another, and uses a stack or recursion (LIFO).

  Both run in O(V + E) time, but BFS generally uses more memory because it stores a whole level of vertices at once. BFS also guarantees the shortest path in an unweighted graph, while DFS does not. BFS is best for shortest-path problems, and DFS is best for cycle detection, topological sorting, and backtracking.

COMPUTER ENGINEERING

QUEZON CITY

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.